

Tarea 01. Desarrollo en PHP

CURSO Y CICLO: 3º DAW Semipresencial.

MÓDULO: Diseño web en entorno servidor.

Título: Desarrollo de una pequeña aplicación web en PHP.

Objetivo:

El alumnado desarrollará una pequeña aplicación web que dará solución al problema planteado teniendo en cuenta, al menos, los requisitos indicados.

Contenido:

1. Elementos del lenguaje PHP.
 - a. Generación de código HTML.
 - b. Cadenas de texto.
 - c. Funciones relacionadas con los tipos de datos.
 - i. Funciones de Fecha I.
 - ii. Funciones de Fecha II.
 - d. Variables especiales de PHP.
2. Estructuras de control.
 - a. Condicionales.
 - b. Bucles.
3. Funciones.
 - a. Inclusión de ficheros externos.
 - b. Ejecución y creación de funciones.
 - c. Argumentos.
4. Tipos de datos compuestos.
 - a. Recorrer arrays (I).
 - b. Recorrer arrays (II).
 - c. Funciones relacionadas con los tipos de datos compuestos.
5. Formularios web.
 - a. Procesamiento de la información devuelta por un formulario web.
 - b. Validación de formularios web en PHP.
 - c. Lectura de datos desde archivos en el servidor.

Resultados de Aprendizaje:

- **RA 3:** Escribe bloques de sentencias embebidos en lenguajes de marcas, seleccionando y utilizando las estructuras de programación.
- **RA 4:** Desarrolla aplicaciones Web embebidas en lenguajes de marcas analizando e incorporando funcionalidades según especificaciones.

Tarea 01. Desarrollo en PHP

Enunciado:

En la reunión de la Comunidad de Vecinos te han elegido presidente. El administrador te plantea la existencia de varios vecinos que tienen cuotas atrasadas. En la actualidad hay una hoja de cálculo donde se almacenan algunos datos de los vecinos, las cuotas pagadas y las no pagadas. Dicha hoja de cálculo presenta distintos problemas, aunque el más importante es la dificultad en la comprensión de los datos mostrados y calculados.

Para resolver definitivamente este problema, se te ha ocurrido realizar una aplicación web que facilite la gestión de las cuotas y evitar el uso de esa hoja de cálculo, además darás solución a la reivindicación de los vecinos que desean consultar sus datos de manera ágil, sin necesidad de realizar una solicitud previa y la espera de varios días a ser respondida.

Se debe programar una aplicación web con los siguientes requisitos:

- Tres roles o usuarios diferentes para el acceso controlado al aplicativo que presentan las siguientes características:
 - El usuario o rol vecino: sólo podrá acceder a ver sus datos.
 - El usuario o rol presidente: podrá ver todos los datos de los vecinos y realizar algunas acciones.
 - El usuario o rol administrador: podrá tener el control total de la aplicación web.
- Se controlará los accesos directos a las páginas php de tal forma que será necesario haber pasado por el formulario de acceso antes de realizar cualquier operación.
- Todos los datos relativos a la comunidad de vecinos serán almacenados en un fichero con extensión .dat (en nuestro caso será vecinos.dat).
- Los datos a ser almacenados serán los siguientes:
 - Nombre y Apellidos del vecino.
 - DNI.
 - Número de teléfono.
 - Correo electrónico.
 - Vivienda: donde se incluya el bloque, piso y la letra.
 - Fecha de alta en la comunidad.
 - Cuotas pagadas.

Tarea 01. Desarrollo en PHP

- Cuotas pendientes.
- Fecha de la última cuota pagada.
- Consideraciones generales:
 - Un vecino tiene una vivienda asignada.
 - No puede haber dos vecinos en una misma vivienda.
 - Un vecino puede tener más de una vivienda.
 - Puede haber tantos vecinos como viviendas haya en la comunidad.
 - El presidente es un vecino.
 - Sólo puede haber un presidente de la comunidad.
 - El administrador pertenece a la gestoría y puede no ser un vecino.
 - Un vecino siempre puede cambiar la contraseña.
 - El usuario tendrá un mensaje de bienvenida a la aplicación indicando su nombre y rol definido.
- Cada vez que se dé de alta un nuevo vecino se tendrá en cuenta lo siguiente:
 - Nombre, Apellidos y DNI no pueden ser valores en blanco.
 - El teléfono y correo electrónico se pueden modificar después de haber sido dado de alta como vecino en la comunidad por lo que su valor puede ser vacío.
 - La fecha de alta en la comunidad no puede ser posterior a la fecha de la última cuota pagada. Su valor por defecto será la fecha en la que el administrador le da de alta, se podrá cambiar por la fecha en la que comienza a ser vecino en la vivienda.
 - Las cuotas pendientes deben ser calculadas, se paga una cuota por mes, por lo que habrá tantas cuotas pendientes como meses entre la fecha de la última cuota pagada y el primer día del mes en curso (o día que se haya aprobado por la comunidad).
- Cada vez que se realice un acceso a la aplicación, ésta se comportará de la siguiente forma:
 - Para consultar los datos los vecinos deben introducir su usuario y contraseña. Accederán sólo a sus datos sin posibilidad de modificarlos.
 - El presidente, una vez accedido correctamente, podrá ver todos los datos de los vecinos y realizar modificaciones de las cuotas (esto es, cuotas pagadas y fecha de la última cuota pagada). Tener en cuenta que, si actualizas la cuota pagada, la fecha de la última cuota pagada también debe ser actualizada y viceversa.

Tarea 01. Desarrollo en PHP

- El administrador, una vez accedido correctamente, puede dar de alta, baja y modificar datos de los vecinos (esto es, teléfono, correo electrónico, vivienda y cuotas).

La práctica consistirá en la elaboración de varios ficheros .php cuya ejecución cumpla con, al menos, los puntos indicados anteriormente. Se deberá tener en cuenta las buenas prácticas en programación.

Se valorará positivamente cualquier mejora aportada por el alumnado debidamente explicada en el documento de entrega.

Se realizará la entrega un único .zip del código y el documento memoria desarrollados. El zip contendrá una carpeta llamada **code** donde se incluirán todos los ficheros .php desarrollados. Esa carpeta, llevada a cualquier servidor Apache deberá ejecutar el aplicativo desarrollado (evitando el uso de archivos y directorios innecesarios no codificados por el alumnado).

La memoria a desarrollar será incluida en ese zip por fuera de la carpeta *code*. Esta memoria explicará cada uno de los puntos indicados en el índice, así como el correspondiente código desarrollado por el alumnado explicando todos los puntos solicitados en el alta, baja y modificado de vecinos. Además, se debe explicar la estructura de ficheros .php utilizada (por ejemplo, en el fichero login.php se ha codificado para ... conteniendo el formulario de ... y realizando las comprobaciones de ...).

A su vez, será necesario la incorporación de varias pantallas que simulen la ejecución del aplicativo con los tres roles definidos. Y, finalmente, una pantalla con el fichero .dat de partida y con el resultado del fichero .dat una vez realizada la simulación.

El nombre del archivo será el siguiente:

Zip: 3DAW_DWS_<NombreApellidos>_UT02_T01

Ejemplo de la estructura del zip:

3DAW_DWS_JuanAntonioPerez_UT02_T01.zip

3DAW_DWS_JuanAntonioPerez_UT02_T01.pdf

code

login.php

logout.php

proceso.php

css/

estilo.css

Puede haber tantos archivos .php y directorios de código, como sea necesario para su ejecución en el

Tarea 01. Desarrollo en PHP

servidor de Apache. Los archivos .php deben poder ser visualizados con cualquier herramienta de texto.

En el desarrollo de la memoria se deben respetar las normas de presentación de las tareas y de las pantallas aportadas.

Criterios de evaluación implicados:

- Se han utilizado mecanismos de decisión en la creación de bloques de sentencias.
- Se han utilizado bucles y se ha verificado su funcionamiento
- Se han utilizado arrays para almacenar y recuperar conjuntos de datos.
- Se han creado y utilizado funciones.
- Se han utilizado formularios Web para interactuar con el usuario del navegador Web.
- Se han empleado métodos para recuperar la información introducida en el formulario.
- Se han añadido comentarios al código.
- Se han identificado los mecanismos disponibles para el mantenimiento de la información que concierne a un cliente Web concreto y se han señalado sus ventajas.
- Se han utilizado mecanismos para mantener el estado de las aplicaciones web.
- Se han utilizado mecanismos para almacenar información en el cliente web y para recuperar su contenido.
- Se han identificado y caracterizado los mecanismos disponibles para la autenticación de usuarios.
- Se han escrito aplicaciones que integren mecanismos de autenticación de usuarios.

Rúbrica (10 puntos):

Cuando se indica Generación correcta de un formulario se valora también la comprobación de datos que se vayan a introducir en este formulario, es decir, si el nombre de un vecino no puede ser blanco deberá indicar un error en caso de que se trate de enviar el formulario, el campo DNI, correo electrónico, teléfono tienen que ser correctos, es decir, un teléfono no puede contener letras.

Generar la estructura de la página PHP correctamente (aplicación de buenas prácticas).	1
Mostrar los datos de los vecinos (archivo .dat) según los roles definidos.	1
Generación correcta del formulario de acceso. Comprobación de datos.	0.25
Generación correcta del formulario de alta de un nuevo vecino. Comprobación de datos.	0.5

Tarea 01. Desarrollo en PHP

Generación correcta del formulario de cambio de contraseña. Comprobación de datos.	0.25
El correcto uso de variables super globales (\$ POST, \$ GET, \$ SESSION, etc.)	0.25
Correcto tratamiento y formato adecuado de los campos de fechas.	0.25
Correcta ejecución del alta de un vecino.	0.75
Correcta ejecución del borrado de un vecino.	0.5
Correcta modificación de los datos de un vecino. Teléfono	0.125
Correcta modificación de los datos de un vecino. Email	0.125
Correcta modificación de los datos de un vecino. Vivienda	0.125
Correcta modificación de los datos de un vecino. Cuotas	0.375
Correcto uso de arrays asociativos.	0.5
Correcto uso de funciones.	1
Mejoras aportadas, debidamente explicadas y justificadas.	1
Introducir comentarios en el código y cuidado del diseño de la página.	1
Respetar las normas de presentación de las tareas.	1
Total	10